

Self-Evolving Agents for Physical AI Evaluation

From Static Benchmarks to Living Evaluation Systems for World Models and Robotic Foundation Models

Anonymous Author(s)

Abstract

Physical AI systems increasingly combine world models, simulation, and vision-language-action policies to perceive, predict, and act in the physical world. However, evaluation remains largely static, fragmented, and weakly calibrated to real-world deployment. Fixed benchmarks can saturate, miss emerging failure modes, and fail silently as model capabilities shift. We propose SEPA-Eval, a self-evolving evaluation architecture for Physical AI. SEPA-Eval continuously generates new embodied tasks, evaluates world models and robot policies jointly, mines failures and sim-real disagreements, calibrates critic ensembles with real-world outcomes, and updates benchmark distributions over time. The architecture centers on an evaluation memory that stores tasks, rollouts, predictions, plans, critic judgments, failures, explanations, metrics, and human feedback. We argue that the central metric for Physical AI evaluation should be predictive validity: whether evaluation results forecast real-world robot behavior and reveal the next frontier of failures. This white paper outlines the architecture, metric taxonomy, research agenda, and implementation roadmap for living evaluation infrastructure for embodied AI.

1 Introduction

Physical AI is entering a new phase. World models can generate plausible futures, simulators can produce large-scale synthetic robot experience, and vision-language-action (VLA) models are becoming general-purpose robotic policies. Yet evaluation remains largely static: fixed task lists, fixed simulators, fixed metrics, and fixed success definitions. This creates a dangerous mismatch. A model can appear to improve on a benchmark while the benchmark itself becomes less informative, less discriminative, or less predictive of deployment behavior.

Consider a tabletop robot that reliably completes a benchmark placement task: move a cup into a tray. In deployment, the same policy fails when the cup is slightly wet, the tray wall is a few centimeters closer, and glare hides the contact point. A video world model predicts a smooth successful placement; the simulator reports success because its friction model is too forgiving; the real robot tips the cup and then repeats the same unsafe grasp. A static benchmark records this as an isolated failure, if it records it at all. A living evaluator should instead ask what this failure reveals: which physical factor mattered, which prediction was wrong, which critic missed the risk, and what new regression test should be added.

This white paper proposes a new evaluation paradigm: *self-evolving agents for Physical AI evaluation*. Instead of treating evaluation as a one-time benchmark, we propose an agentic system that observes model behavior, mines failures, generates new tests, validates them in simulation and the real world, updates the evaluation suite, and tracks when existing evals become obsolete. This framing is motivated by recent arguments that evaluation protocols themselves break as model capabilities shift [13], and extends

that argument to embodied systems where failures can arise from contact dynamics, sensing, language grounding, planning, control, safety, and sim-to-real mismatch.

We focus on two coupled model families. The first is *world models*: video/action-conditioned predictors, neural simulators, object-centric dynamics models, and foundation world models. The second is *robotics models*: VLA policies, diffusion or flow-based policies, hierarchical controllers, and multi-embodiment robot foundation models. Our core claim is that these systems should not be evaluated separately. A world model is useful only if it preserves the physical and causal structure needed for embodied action. A robot policy is trustworthy only if it remains robust under meaningful physical variation.

SEPA-Eval is best understood as an evaluation protocol and infrastructure pattern, not a single benchmark or model. Its novelty is the integration of four ideas that are usually separated: adaptive test generation, trace-level evaluation memory, joint world-model and robot-policy assessment, and promotion rules tied to real-world predictive validity. In this framing, a benchmark item is not merely a task; it is a provenance-rich record of why the task matters, which failures it exposes, and how well it predicts behavior outside the benchmark.

Physical AI should not be evaluated only by asking whether a robot succeeds on today's tasks. It should be evaluated by asking whether the evaluation system can continuously discover tomorrow's failures.

2 Why Static Physical AI Evaluation Fails

Evaluation is often treated as a fixed measurement problem: choose a benchmark, define metrics, run models, and compare scores. This paradigm is insufficient for Physical AI. Embodied agents interact with the world through perception, language, planning, control, and contact-rich dynamics. Their failures are rarely one-dimensional. A robot may understand the instruction but misjudge an affordance; predict free-space motion correctly but fail at contact; succeed under ideal lighting but fail under reflection, transparency, clutter, or a slight camera shift; or pass a simulator benchmark by exploiting simulator artifacts.

Static benchmarks fail in at least three ways.

Saturation. Once models exceed the difficulty level of a benchmark, the benchmark no longer separates frontier systems.

Misalignment. A benchmark may reward shallow success while missing what matters for deployment: safety, recovery, robustness, calibration, and predictive validity.

Silent failure. Scores may continue to look meaningful even when they no longer predict real-world outcomes.

The practical symptom is not merely that a leaderboard becomes stale. It is that the evaluation system stops producing useful scientific information. Teams can no longer tell whether a score increase reflects a more capable policy, a better-tuned simulator interface,

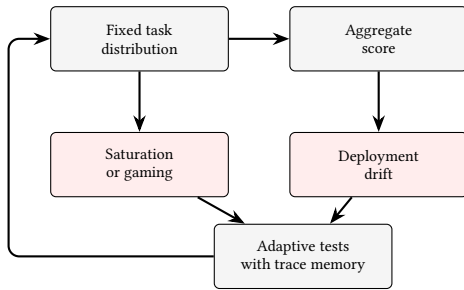


Figure 1: Static benchmarks compress behavior into scores, while living evaluation routes saturation, metric gaming, and deployment drift back into new tests.

memorization of common task templates, or exploitation of underspecified success criteria. For Physical AI, this is particularly problematic because deployment failures are often rare combinations of ordinary factors: a slightly reflective object, an instruction with a hidden constraint, a shifted camera, a marginal grasp, and a recovery policy that is never invoked in the benchmark.

Static evaluation also underweights temporal structure. A rollout can be successful until the final contact event, safe until the robot begins recovery, or physically plausible for the first few frames before a world model loses object permanence. Useful evaluation therefore needs to preserve traces, intermediate decisions, counterfactual branches, and failure precursors rather than collapsing behavior into a single final score.

These failure modes are especially dangerous in robotics because embodied failures can be costly, unsafe, and difficult to reproduce. The next generation of evaluation should therefore be adaptive: it should observe failures, generate tests around them, validate those tests, and update the benchmark distribution.

3 Related Work and Resource Landscape

3.1 VLA and robot foundation models

VLA models map visual observations and language instructions into robot actions. RT-2 introduced a VLA recipe that incorporates web-scale vision-language pretraining directly into robotic control [2]. OpenVLA provides a widely used open-source 7B-parameter VLA trained on large-scale robot demonstrations [4]. π_0 proposes a vision-language-action flow model for general robot control, trained across multiple dexterous robot platforms [1]. Industrial systems such as Isaac GR00T N1 further signal movement toward large robot foundation models for humanoids [10].

These advances intensify the need for stronger evaluation. A fixed set of tabletop manipulation tasks cannot fully test physical reasoning, long-horizon execution, cross-embodiment transfer, safety, or robustness under environmental variation.

3.2 World-model evaluation

World models can serve as synthetic data engines, policy evaluators, action planners, safety forecasters, counterfactual generators, and learned simulators. WorldArena emphasizes that embodied world models should be evaluated not only by perceptual quality but also by functional utility, including use as data engines, policy

evaluators, and planners [14]. WorldModelBench highlights world-model-specific violations such as instruction-following and physics-adherence failures [6]. Physion and Physion-Eval focus on physical realism and human physical reasoning in generated videos [11, 16]. Morpheus evaluates physical conservation properties in generated video [15].

For robotics, the key point is that visual realism is not enough. RoboWM-Bench argues that visually plausible generated behavior may still fail when converted into executable robot actions, and proposes embodiment-grounded evaluation of world models in manipulation [3]. WorldEval studies whether world models can serve as scalable proxies for real-world robot-policy evaluation [8], while dWorldEval extends this direction with discrete diffusion world models for scalable policy evaluation [7].

3.3 Embodied simulation and task benchmarks

Large-scale simulation benchmarks provide the substrate for evolving evaluation distributions. RoboCasa provides realistic household environments and tasks for generalist robot learning [9]. RoboCasa365 expands toward large-scale household mobile manipulation, multi-task learning, foundation model training, and lifelong learning [12]. BEHAVIOR-1K introduces 1,000 human-centered everyday activities across realistic scenes with rich physical and semantic object properties [5]. Such benchmarks are essential, but we argue that they should become sources of evolving test distributions rather than fixed leaderboards.

3.4 What changes in a living evaluator

SEPA-Eval builds on familiar ideas including active learning, adversarial evaluation, procedural task generation, red teaming, simulator validation, and continuous regression testing. The difference is the unit of optimization. Active learning usually selects data to improve a model; SEPA-Eval selects evidence to improve an evaluation distribution. Red teaming often produces isolated failures; SEPA-Eval turns failures into validated benchmark artifacts with provenance and recurrence tracking. Simulator validation often asks whether simulation is realistic; SEPA-Eval asks which simulator or world-model results are predictive for which task families, embodiments, and physical phenomena.

This shift matters because Physical AI evaluation is not only a scoring problem. It is an infrastructure problem: how to preserve behavioral evidence, compare predictions against reality, detect benchmark obsolescence, and decide which new tests deserve institutional status. The contribution of SEPA-Eval is therefore not a new metric alone, but a closed-loop protocol for keeping metrics, tasks, critics, and simulators accountable to deployment outcomes.

4 Architecture Overview

Figure 2 summarizes the proposed architecture. The system contains seven major components: subject models, task and scenario evolution, world-model evaluation, VLA/robot evaluation, critic ensembles, sim-real calibration, and evaluation memory.

The design principle is that evaluation outputs are not the end of the process. They feed back into the next generation of tasks, critics, metrics, and model-improvement signals. The evaluator is itself an agent: it observes behavior, forms hypotheses about weaknesses,

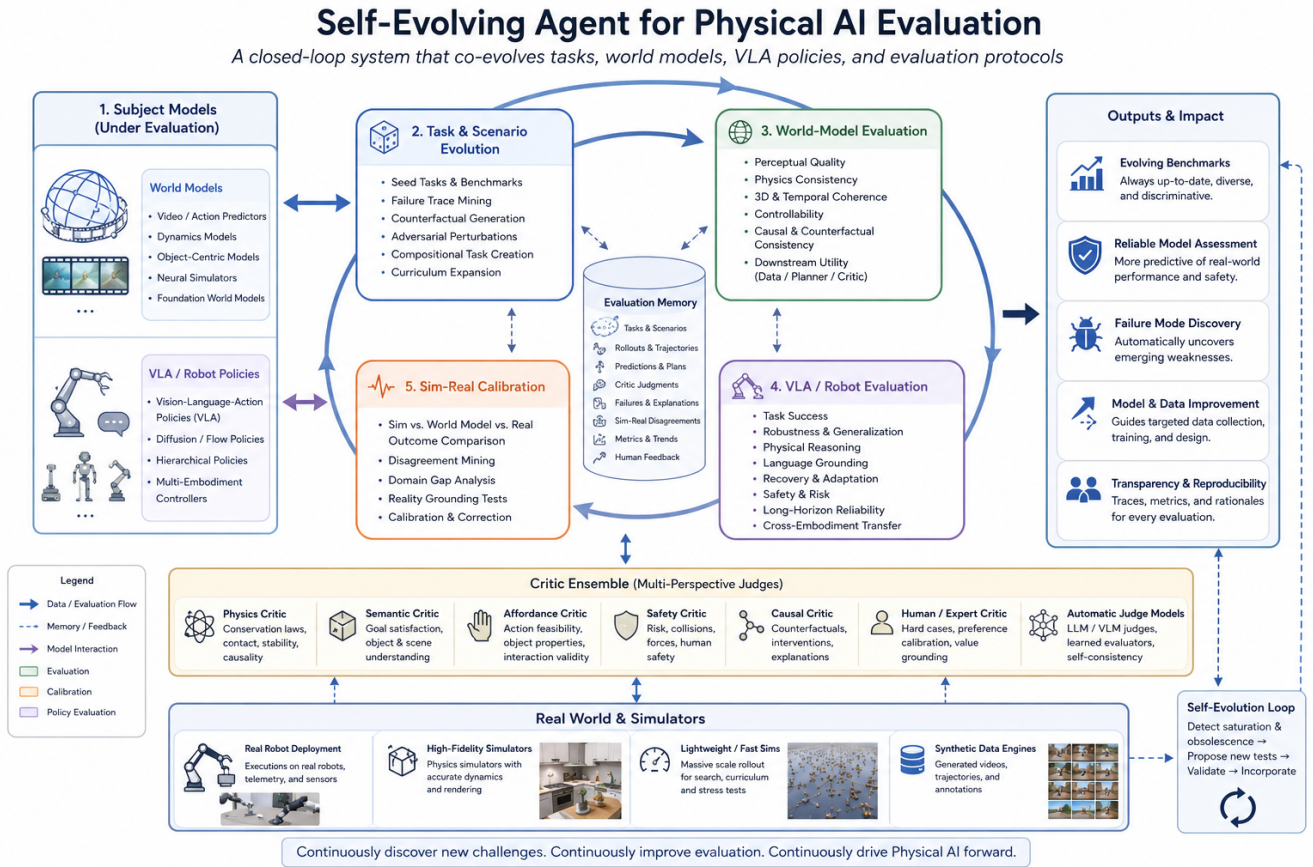


Figure 2: Self-evolving agent for Physical AI evaluation. The architecture treats evaluation as a closed-loop system that co-evolves tasks, world models, robot policies, critic ensembles, simulators, real-world rollouts, and evaluation protocols. The central evaluation memory stores tasks, rollouts, predictions, plans, critic judgments, failures, metrics, and human feedback.

designs experiments, runs them across world models and robots, and updates the evaluation suite.

The architecture is intentionally model-agnostic. A subject model can be a closed-source VLA policy, an open robot foundation model, a learned simulator, a video world model, or a conventional controller wrapped in a planning stack. What matters is that each run produces enough evidence for comparison: initial scene state, instruction, action trace, predicted future, observed future, success labels, critic scores, uncertainty, and provenance. This makes the evaluator a common measurement layer across otherwise heterogeneous robotics stacks.

The running cup-and-tray example illustrates the intended loop. The original failure is stored as a trace, including the wet cup, tray geometry, glare, robot actions, predicted world-model future, simulator outcome, real video, contact event, and critic judgments. The evaluator then creates paired variants: dry versus wet cup, wide versus narrow tray, clear view versus glare, and alternative grasp strategies. It validates that each variant is physically plausible and solvable by at least one competent policy or human teleoperator. It promotes the variants that expose reproducible, discriminative, deployment-relevant failures, while archiving variants that are

redundant or unrealistic. Subsequent model releases are evaluated against this family of tests, and the system tracks whether success on them predicts safer real-world placements.

SEPA-Eval is not intended to replace fixed benchmarks. Stable benchmark slices remain necessary for longitudinal comparison, public reporting, and reproducibility. The point is to surround these fixed slices with an adaptive layer that detects when they are saturated, discovers new failure modes, and provides a principled path for promoting new tests without erasing historical comparability.

5 Subject Models Under Evaluation

5.1 World models

World models include video prediction models, action-conditioned video models, object-centric dynamics models, neural simulators, learned physical predictors, foundation world models, and hybrid simulator-neural models. They should be evaluated both as generative models and as functional components in the robotics stack.

In a self-evolving setting, world models are not only evaluated after training. They also participate in evaluation generation. A world model can propose likely failure futures, create counterfactual

continuations, identify physically ambiguous states, and estimate whether an expensive real-world rollout is worth running. The evaluator must therefore track two roles separately: the world model as a *subject* being measured and the world model as a *tool* used to generate or filter test cases. Mixing these roles without provenance can create circular evaluation, where a model is rewarded for satisfying assumptions that it helped create.

A world model used for robotics should satisfy five forms of validity:

- (1) **Perceptual validity**: does the generated future look realistic?
- (2) **Physical validity**: does it obey relevant physical constraints?
- (3) **Causal validity**: does it correctly predict effects of actions and interventions?
- (4) **Behavioral validity**: does it predict robot policy success or failure?
- (5) **Deployment validity**: does it correlate with real-world robot outcomes?

5.2 VLA and robot policies

Robot policies include VLA systems, diffusion or flow-based action policies, hierarchical planners, cross-embodiment controllers, mobile manipulation policies, and humanoid policies. The evaluation system should treat policies not only as black-box success-rate generators, but as behavioral systems with traces, decisions, recoveries, and failure modes.

For each policy rollout, the evaluator should record the observation stream, language context, intermediate plans if available, action commands, controller state, safety overrides, force or contact events, human interventions, and recovery attempts. These traces make it possible to distinguish a policy that succeeds robustly from one that reaches the final state through unsafe or brittle behavior. They also allow failure reuse: a failed rollout can become a seed for new world-model prompts, simulator variants, critic calibration examples, and real-world regression tests.

6 Task and Scenario Evolution

The task evolution module generates new evaluation cases from multiple sources.

6.1 Seed tasks and benchmarks

The system begins from existing benchmarks, task suites, and real-world deployment scenarios. These provide initial coverage but should not be treated as sufficient.

Seed tasks should be stored with explicit metadata: task intent, required objects, physical skills, language phenomena, safety constraints, embodiment assumptions, simulator assets, and expected success predicates. This metadata lets the evaluator generate targeted variants rather than random noise. For example, a seed task involving pouring can be expanded along container geometry, liquid level, viscosity, grip pose, occlusion, and spillage tolerance.

6.2 Failure trace mining

Every failed rollout is a source of new tests. The agent extracts the attempted goal, physical context, action preceding failure, whether the model perceived the failure, whether recovery was attempted,

and whether the cause was perception, planning, control, physics, language grounding, or embodiment mismatch.

Failure mining should preserve the smallest reproducible slice of the rollout. If a twenty-step task fails because the robot nudges a bowl while reaching for a spoon, the benchmark candidate should include both the full long-horizon task and a distilled micro-scenario around the reach, occlusion, and contact event. The long task tests planning and recovery; the micro-scenario tests the local physical competence that caused the failure.

6.3 Counterfactual generation

The agent generates counterfactual variants: same task with different object pose; same instruction with distractors; same scene with altered lighting; same manipulation with changed mass or friction; same goal with a different tool; same success case with added human interruption.

Counterfactuals are most useful when they isolate causal factors. A good mutation changes one hypothesis at a time: pose while preserving instruction, language while preserving scene, friction while preserving geometry, or embodiment while preserving goal. Bundled perturbations are still valuable for stress testing, but they are harder to attribute. The evaluator should therefore maintain paired tests that reveal whether a model is sensitive to the intended physical or semantic change.

6.4 Adversarial physical perturbation

The system searches for minimal physical changes that cause failure, such as small camera shifts, occlusion, reflective surfaces, transparent objects, unstable stacks, slippery contact, deformable objects, cables, clutter, and ambiguous language.

6.5 Curriculum expansion

When models begin to solve a task family, the system expands difficulty: longer horizon, more objects, more distractors, less observability, stricter safety constraints, more ambiguous instructions, higher physical precision, and more recovery requirements.

Promotion should be conservative. A generated test becomes part of the benchmark distribution only when it satisfies criteria such as those in Table 1. This prevents the evaluator from drifting toward strange adversarial cases that are easy to generate but irrelevant to deployment.

7 World-Model Evaluation

This section combines perceptual, physical, causal, and functional metrics for world-model evaluation.

7.1 Perceptual and temporal quality

Perceptual metrics ask whether the generated future looks plausible. They include visual fidelity, temporal coherence, frame consistency, object appearance stability, scene consistency, and camera motion realism. These metrics are useful but insufficient.

The evaluator should prefer diagnostics that expose temporally localized defects. Instead of only reporting a sequence-level realism score, it should identify when an object disappears, when contact begins, when a support relation changes, and when the generated future diverges from an observed or simulator rollout.

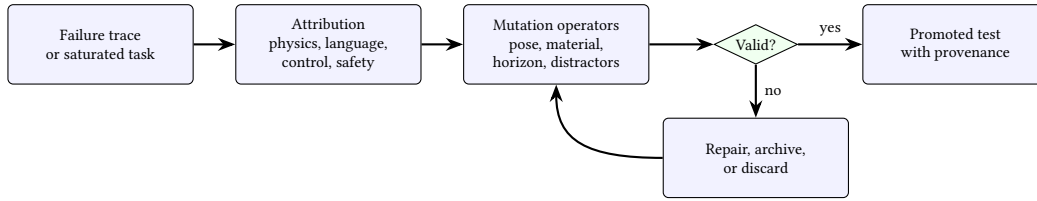


Figure 3: Trace-to-benchmark distillation. The evaluator converts failures and saturated tasks into attributed mutations, validates them, and promotes only reproducible, meaningful, non-redundant tests.

Table 1: Promotion criteria for turning generated scenarios into benchmark items.

Criterion	Meaning	Example in the cup-and-tray task
Physical validity	The scene and outcome obey plausible geometry, material, and contact constraints.	Wet-cup friction values are within measured or documented ranges.
Solvability	A competent policy, teleoperator, or planner can complete the task safely.	A human teleoperator can place the cup using a slower approach and adjusted grasp.
Reproducibility	The failure or capability distinction recurs across repeated trials or seeds.	The same policy repeatedly tips the cup under glare and narrow-tray conditions.
Non-redundancy	The scenario adds information beyond existing benchmark cases.	The test isolates slip at tray contact rather than duplicating generic grasp failure.
Discriminative power	The scenario separates models, policies, or critic configurations.	A policy with force-aware recovery succeeds while a vision-only policy fails.
Deployment relevance	The scenario reflects a credible real-world condition or safety concern.	Condensation, glare, and tight tray clearance occur in household serving tasks.
Predictive validity	Performance on the scenario correlates with real-world outcomes or known incident classes.	Passing the test predicts fewer cup-tip failures in real robot trials.

These event-level annotations make perceptual metrics useful for robotics debugging.

7.2 Physical consistency

Physical consistency tests whether the model obeys action-relevant constraints: object permanence, collision consistency, contact stability, gravity and support, rigid-body motion, deformable-object behavior, liquid behavior, friction and slip, containment, causality, and reversibility.

Physical consistency should be measured at multiple granularities. Some tests are local, such as whether a grasped object moves with the gripper. Others are relational, such as whether a cup remains supported by a tray after placement. Still others are counterfactual, such as whether increased friction reduces slip in otherwise identical scenes. A strong world model should behave consistently across all three levels.

7.3 3D and spatial consistency

Robotics requires more than 2D video realism. The system should test depth consistency, object-pose stability, view consistency, occlusion handling, spatial relations, affordance geometry, and reachability.

7.4 Controllability

A world model used for robotics must follow action conditions. It should not merely generate plausible futures; it should generate futures that reflect commanded actions. Useful tests include

same-scene/different-action comparisons, action ablations, counterfactual action comparisons, trajectory-conditioned generation, and success/failure prediction under alternative plans.

7.5 Downstream robot utility

The most important question is whether the world model helps robotics. The system should evaluate whether the world model can generate synthetic data that improves policies, rank policies similarly to real-world evaluation, support action planning, detect unsafe futures, identify failure causes, improve recovery policies, and reduce real-world trial cost.

This requires interventional evaluation rather than passive scoring. The evaluator should compare policy training with and without synthetic data, planning with and without world-model rollouts, and safety filtering with and without predicted futures. A world model that produces visually appealing videos but does not improve any downstream decision should receive a low embodied-utility score.

8 VLA and Robot Policy Evaluation

Robot policy evaluation should go beyond task success.

8.1 Task success

Success should be measured at multiple levels: binary completion, partial progress, time to completion, number of interventions, action efficiency, energy or force efficiency, collateral damage, and final-state quality.

The system should also distinguish *clean success* from *fragile success*. Clean success satisfies the task without unnecessary contact, unsafe forces, repeated retries, hidden human assistance, or lucky recoveries. Fragile success reaches the final state but leaves evidence that the policy is near a failure boundary. This distinction is important for regression testing because many deployment incidents begin as fragile successes in evaluation.

8.2 Robustness and generalization

The system should test whether success survives distribution shift: object pose changes, object shape changes, material changes, lighting changes, camera shifts, clutter, distractors, paraphrases, unseen object categories, unseen scenes, and different embodiments.

8.3 Physical reasoning

The system should evaluate whether the robot handles support, stability, containment, force, friction, deformation, tool use, collision, reachability, occlusion, and irreversibility.

8.4 Language-grounded action

The policy should bind language to objects, attributes, relations, constraints, subgoals, safety requirements, temporal order, and exception conditions.

8.5 Recovery and adaptation

Deployed robots must handle failure. The evaluation should measure failure detection, self-correction, retry strategy, replanning, asking for help, safe fallback, and graceful degradation.

Recovery metrics should be grounded in the failure moment. The evaluator should ask whether the robot noticed the failure, whether its diagnosis was correct, whether the next action reduced risk, and whether the recovery preserved task intent. A policy that retries the same bad grasp five times is different from one that changes viewpoint, selects a safer grasp, or asks for clarification.

8.6 Safety and risk

Safety should be a first-class metric. Tests should include collision risk, excessive force, dropping or spilling, damaging fragile objects, unsafe proximity to humans, sharp or hot objects, irreversible actions, unsafe exploration, and out-of-distribution uncertainty.

9 Critic Ensemble

No single metric can judge Physical AI. SEPA-Eval uses a critic ensemble:

- **Physics critic:** gravity, support, contact, collision, conservation, deformation, and causal continuity.
- **Semantic critic:** task completion, object correctness, constraint satisfaction, and instruction following.
- **Affordance critic:** graspability, tool appropriateness, reachability, and manipulation feasibility.
- **Safety critic:** collisions, spills, breakage, human proximity, unsafe forces, and irreversible operations.
- **Causal critic:** interventions, failure causes, and counterfactual explanations.

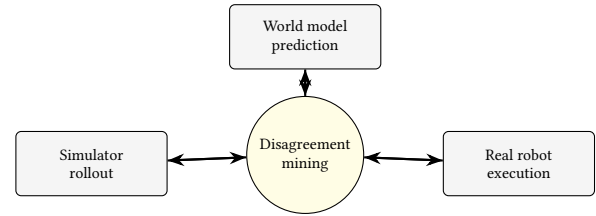


Figure 4: Sim-real calibration compares predictions, simulation, and real executions. Disagreements become calibration data, simulator-improvement targets, and new benchmark cases.

- **Human/expert critic:** calibration, edge cases, ambiguous tasks, and high-stakes judgments.
- **Automatic judge models:** scalable LLM/VLM-based evaluation calibrated against humans, physics engines, and real outcomes.

The critic ensemble should produce not only scores, but also explanations and uncertainty estimates.

Critics should be calibrated against one another and against outcome data. When a semantic critic says the task is complete but a physics critic reports unstable contact, the evaluator should preserve the disagreement instead of averaging it away. Such disagreement often marks the most useful cases: visually successful but unsafe rollouts, plausible world-model videos with impossible contacts, or policies that satisfy a literal instruction while violating an implicit safety constraint.

The ensemble should also maintain critic-specific reliability estimates. A VLM judge may be strong at object identity but weak at force, a physics engine may be strong at rigid contact but weak at deformable material, and a human expert may be accurate but expensive and inconsistent under ambiguous instructions. Reliability estimates let the evaluator route high-stakes cases to the right judges and avoid over-trusting a single scoring channel.

10 Sim-Real Calibration

The sim-real calibration loop compares outcomes across world-model predictions, simulator rollouts, and real robot executions. The most valuable cases are disagreements:

- world model predicts success, simulator predicts success, real robot fails;
- simulator predicts failure, real robot succeeds;
- world model predicts safety, real rollout is unsafe;
- policy succeeds in one embodiment but fails in another;
- synthetic data improves simulator success but hurts real-world performance.

These disagreements define the frontier of evaluation uncertainty. The agent should mine them and turn them into new tests. Calibration should measure rank correlation between simulated and real policy performance, task-level success correlation, failure-mode overlap, trajectory similarity, contact-event similarity, distribution-shift sensitivity, confidence calibration, and real-world predictive validity.

Operationally, calibration should be stratified by task family, object class, embodiment, and physical phenomenon. A simulator

may be predictive for rigid pick-and-place but unreliable for cables or liquids; a world model may rank policies well in uncluttered tabletop scenes but fail under reflective surfaces. The evaluator should report these validity regions explicitly so teams know where simulated results can be trusted and where real-world sampling remains necessary.

11 Evaluation Memory

Evaluation memory stores task definitions, scene configurations, object metadata, robot embodiments, policy versions, world-model versions, simulator settings, rollouts, videos, trajectories, actions, rewards, success labels, critic judgments, human feedback, failure explanations, sim-real disagreements, benchmark saturation history, and metric drift.

This memory lets the system answer: which tasks are saturated; which tasks still discriminate frontier models; which failures recur across model families; which simulator results correlate with reality; which world-model metrics predict downstream utility; and which generated tests should be promoted into the official benchmark.

The memory should be organized around immutable evaluation records. Each record links a task specification, environment configuration, model version, run configuration, outputs, judgments, and derived annotations. New critics or success predicates can be run later, but the original evidence should remain unchanged. This allows retrospective analysis when a benchmark saturates, when a critic is found to be biased, or when a real-world incident reveals that an old failure pattern was underweighted.

A useful memory also needs indexing for retrieval. Engineers should be able to query for all failures involving transparent containers, all tasks where simulation overpredicted success, all policies that improved average success while increasing unsafe contact, or all world-model predictions that were visually plausible but physically invalid. These retrieval capabilities turn evaluation history into a debugging and research asset rather than a passive log archive.

Table 2 sketches the minimum record structure. The schema is intentionally evidence-centered: generated summaries, critic scores, and benchmark decisions are derived from auditable artifacts rather than replacing them.

12 The Self-Evolution Loop

SEPA-Eval iterates through seven steps.

- (1) **Evaluate:** run world models and robot policies on the current task distribution.
- (2) **Diagnose:** cluster failures into perception, language, physics, contact, control, planning, recovery, safety, sim-real, world-model, or judge failures.
- (3) **Generate:** create task candidates by perturbing scenes, composing subgoals, increasing horizon, varying physics parameters, and replaying failures.
- (4) **Validate:** check physical validity, reproducibility, solvability, diversity, non-redundancy, discriminative power, real-world relevance, and safety relevance.
- (5) **Promote:** add high-value tests to the benchmark distribution.
- (6) **Monitor obsolescence:** track saturation, discriminative power, predictive validity, and metric gaming.

- (7) **Report:** summarize capability frontiers, recurring failures, robustness cliffs, sim-real gaps, safety risks, and recommended next tests.

The loop should run at multiple cadences. Fast simulator and world-model checks can run continuously on every model version. More expensive real-robot calibration can run on a budgeted schedule or be triggered by high-uncertainty disagreements. Human review should be reserved for ambiguous, safety-relevant, or benchmark-promotion cases. This tiered cadence makes the system practical: most tests are cheap, but the evaluator still maintains a real-world anchor.

13 Worked Example: From Failure to Benchmark

The cup-and-tray failure can be converted into a benchmark family through a concrete sequence. First, the evaluator captures the original trace: instruction, scene geometry, wet cup surface, glare, robot action sequence, world-model prediction, simulator rollout, real video, contact event, and final failure label. Second, critic disagreement is recorded: the semantic critic may judge the cup as nearly placed, while the physics and safety critics flag unstable contact and repeated unsafe force. Third, the task evolution module generates paired counterfactuals that isolate likely causes, including dry versus wet cup, wide versus narrow tray, direct view versus glare, and nominal versus force-aware recovery.

Fourth, the evaluator validates these candidates. Unrealistic variants are rejected, redundant variants are archived, and solvable variants are kept only if they expose a reproducible distinction among policies or world models. Fifth, a small subset is promoted as a benchmark family with provenance. The promoted family no longer asks only “can the robot place a cup in a tray?” It asks whether the robot can place a cup under realistic contact uncertainty, whether the world model predicts slip risk, whether the simulator is calibrated for the relevant materials, and whether the policy changes behavior after detecting incipient failure.

Finally, the family is monitored over time. If all frontier policies eventually solve the basic wet-cup case, the evaluator can retain it as a regression test while expanding to tighter clearances, more ambiguous visual conditions, or human-proximity constraints. If the case stops predicting real-world failures, its validity region is narrowed or it is retired from headline scoring. The benchmark is therefore not merely larger; it is more accountable to the evidence that made it useful.

14 Metrics

Table 3 summarizes metric families for world models, robot policies, and the self-evolving evaluator itself.

15 Example Evaluation Scenarios

15.1 Contact-rich manipulation

A robot must place a slippery glass cup into a narrow tray without tipping it. Perturbations include material changes, friction variation, condensation, tray-pose shift, visual reflection, and partial occlusion. Failures include unstable grasping, mispredicted slip,

Table 2: Minimal evaluation-memory record for traceable, revisable evaluation.

Field group	Contents	Why it matters
Task and scene	Instruction, success predicate, scene assets, object properties, physical parameters, safety constraints.	Makes task mutations interpretable and reproducible.
Embodiment and model	Robot embodiment, controller stack, policy version, world-model version, simulator version, calibration settings.	Separates model capability from embodiment and tooling effects.
Run evidence	Observations, actions, predicted futures, simulator rollouts, real videos, trajectories, contact events, interventions.	Preserves the behavioral trace needed for later reanalysis.
Judgments	Automatic critic scores, human labels, uncertainty, disagreement flags, failure attribution, explanations.	Allows multiple judges to be compared and recalibrated.
Provenance and status	Source benchmark or failure, mutation lineage, validation results, promotion decision, retirement history.	Prevents opaque benchmark drift and supports auditability.
Predictive validity	Correlation with real outcomes, validity region, confidence intervals, known counterexamples.	Records where the evaluation can and cannot be trusted.

Table 3: Metric families for self-evolving Physical AI evaluation.

Target	Metric family	Example questions
World model	Visual and temporal realism	Does the generated future look plausible and remain coherent?
	Physical consistency	Are contact, gravity, support, deformation, and friction plausible?
	3D/spatial consistency	Are pose, depth, occlusion, and affordance geometry stable?
	Action controllability	Does the future change correctly under different actions?
	Causal consistency	Do interventions lead to expected consequences?
	Downstream utility	Does the model improve synthetic data, planning, policy ranking, or safety forecasting?
Robot policy	Task success	Does the robot complete the task under constraints?
	Robustness	Does success survive perturbations?
	Generalization	Does it transfer across objects, scenes, and embodiments?
	Physical reasoning	Does it handle support, contact, friction, stability, and tool use?
Evaluator	Recovery and safety	Can it detect failure, replan, and avoid unsafe outcomes?
	Calibration	Does confidence reflect actual success probability?
	Failure discovery rate	How many novel, meaningful failures are found?
	Test promotion quality	What fraction of generated tests become useful benchmark cases?
	Predictive validity	Do evaluation scores predict real-world behavior?
	Saturation detection	Does the agent detect benchmark obsolescence?

collision with the tray wall, overconfident world-model forecasts, and unsafe force.

15.2 Long-horizon kitchen task

A robot must prepare a breakfast setting: retrieve a bowl, spoon, cereal box, and milk; then place them on the table in the correct order. Perturbations include unexpected object locations, distractors, missing objects, ambiguous instructions, human interruption, and partial failures requiring recovery.

15.3 World-model policy ranking

The evaluator rolls out multiple policies in a world model, rolls them out in simulation, executes a subset on real hardware, and

compares ranking correlations. This tests whether the world model can serve as a reliable proxy evaluator.

15.4 Safety-critical counterfactual

A robot must move a hot mug around a human hand near a table edge. Perturbations include human motion, unexpected mug weight, slippery surfaces, and partial occlusion. The key question is whether the policy slows down, increases clearance, or triggers a safe fallback.

16 Research Agenda

16.1 Generating physically meaningful tests

A key challenge is generating tasks that are hard, meaningful, and physically valid. Open questions include how to generate solvable variants, balance diversity and realism, and avoid redundant adversarial examples.

16.2 Calibrating critic ensembles

Automatic judges are useful but dangerous when uncalibrated. We need methods to combine physics critics, VLM judges, simulators, and humans; detect judge failure; estimate uncertainty; and prevent models from gaming learned judges.

16.3 Measuring world-model utility

The central question is whether a world model improves robot evaluation and control. Open questions include which metrics predict real robot performance, when world models can replace real-world evaluation, and how to represent action, contact, and uncertainty.

16.4 Mining sim-real disagreement

Disagreements among world models, simulators, and real robots are scientific signals. We need methods to attribute disagreement to perception, physics, control, embodiment, or task definition, and to use disagreement to update benchmarks and simulators.

16.5 Evaluation memory and trace-to-benchmark distillation

A long-term evaluation system needs persistent memory. Open questions include how to represent traces, distill many failures into compact tests, preserve provenance, and track benchmark obsolescence over time.

16.6 Safety-focused self-evolution

Self-evolving evaluation should prioritize safety-relevant failures. Open questions include how to generate controlled safety-critical scenarios, score irreversible risk, evaluate uncertainty-aware fallback behavior, and decide when a model is unsafe for real-world rollout.

17 Implementation Roadmap

Phase 1: Offline benchmark and trace analysis. Build the first evaluation memory from existing benchmark rollouts, simulator traces, robot videos, VLA logs, world-model predictions, and human labels. Produce a failure taxonomy, task mutation library, critic suite, and reporting dashboard.

Phase 2: Simulator-in-the-loop generation. Connect the evaluator to high-throughput simulation for procedural scene variation, object perturbation, physics parameter randomization, automatic validation, and fast rollouts.

Phase 3: World-model-in-the-loop evaluation. Use world models for action-conditioned prediction, policy ranking, failure forecasting, safety robot prediction, and synthetic data generation.

Phase 4: Real-robot calibration. Select high-value tasks for real-world execution. Compare real rollouts against simulator and

world-model predictions, calibrate critics, and estimate predictive validity.

Phase 5: Continuous evaluation service. Deploy recurring evaluations, automatic benchmark updates, model-version comparisons, safety regression tests, and living leaderboards.

18 Stakeholder Value

For research leaders, self-evolving evaluation provides adaptive infrastructure that keeps pace with rapidly improving VLA and world models. For robot model builders, it provides targeted failure discovery and debugging signals before expensive real-world deployment. For world-model builders, it shifts the evaluation target from video realism to embodied utility. For safety teams, it converts discovered edge cases into regression tests. For platform teams, it becomes shared infrastructure for task generation, rollout storage, critic services, simulator connectors, and real-world calibration. For executives, it is a strategic layer: as models commoditize, high-quality evaluation and failure discovery become durable assets.

19 Discussion and Limitations

A self-evolving evaluator introduces risks. Generated tests may be unrealistic, redundant, or biased toward evaluator-specific artifacts. Automatic judges may be uncalibrated or gamed. Real-world calibration is expensive and may cover only a narrow subset of deployment contexts. The architecture therefore requires explicit validation gates, provenance tracking, uncertainty estimates, human oversight, and safety-first promotion criteria.

Another limitation is that predictive validity may vary by embodiment and task family. A world model that ranks tabletop manipulation policies well may not rank humanoid locomotion policies. The system should therefore maintain per-domain validity estimates rather than a single global trust score.

There is also a governance risk. If a living benchmark is controlled by a small number of labs or vendors, its evaluation memory can become an opaque source of power: it may define which failures matter, which simulators are trusted, and which models appear safe. Public benchmark slices, auditable promotion records, and independent reproduction are therefore essential. A living evaluator should preserve more evidence than it exposes in public leaderboards, but it should still make its promotion logic and validity claims inspectable.

The adaptive loop can also narrow rather than broaden evaluation if it overfits to the failures it already knows how to generate. A system that repeatedly mutates cup-placement failures may miss navigation, human interaction, deformable objects, or long-horizon planning failures. Diversity constraints and periodic human-led scenario review are needed to keep the search space broad.

Finally, predictive validity is not a permanent property. It can decay as robot hardware changes, policies learn to exploit critics, simulators are updated, or deployment contexts shift. This means benchmark maintenance becomes a first-class scientific activity rather than a one-time release. The burden is higher than for static benchmarks, but the alternative is worse: evaluation artifacts that remain stable precisely because they have stopped measuring the frontier.

20 Conclusion

Physical AI evaluation should become a living system. Static benchmarks are necessary but insufficient: they saturate, drift, and fail silently as world models and robot policies improve. We proposed SEPA-Eval, a self-evolving architecture that jointly evaluates world models and robot policies, mines failures across simulation and reality, generates new embodied tasks, calibrates critic ensembles, and updates benchmarks as capabilities change.

The central metric is predictive validity: whether evaluation results forecast real-world robot behavior and reveal the next frontier of failures. The central infrastructure is evaluation memory: a persistent record of tasks, rollouts, predictions, judgments, failures, explanations, and sim-real disagreements. The central research agenda is to transform evaluation from a static leaderboard into a closed-loop scientific instrument for Physical AI.

The immediate call to action is practical. Benchmark builders should publish provenance and promotion criteria, not only task lists. Robotics labs should report where simulated and world-model evaluations predict real outcomes, and where they fail. Model builders should treat failure traces as reusable evaluation assets. Safety teams should convert deployment incidents into calibrated regression tests. If Physical AI is to leave controlled demonstrations and enter open-ended environments, its evaluation infrastructure must evolve at least as quickly as the models it measures.

References

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. 2024. π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164 [cs.RO] <https://arxiv.org/abs/2410.24164>
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818 [cs.RO] <https://arxiv.org/abs/2307.15818>
- [3] Feng Jiang, Yang Chen, Kyle Xu, Yuchen Liu, Haifeng Wang, Zhenhao Shen, Jasper Lu, Shengze Huang, Yuanfei Wang, Chen Xie, and Ruihai Wu. 2026. RoboWM-Bench: A Benchmark for Evaluating World Models in Robotic Manipulation. arXiv:2604.19092 [cs.RO] <https://arxiv.org/abs/2604.19092>
- [4] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246 [cs.RO] <https://arxiv.org/abs/2406.09246>
- [5] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabriel Levine, Wensi Ai, Benjamin Martinez, Hang Yin, Michael Lingelbach, Minjune Hwang, Ayano Hiranaka, Suyay Garlanka, Arman Aydin, Sharon Lee, Jiankai Sun, Mona Anvari, Manasi Sharma, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R. Matthews, Ivan Villa-Renteria, Jerry Huayang Tang, Claire Tang, Fei Xia, Yunzhu Li, Silvio Savarese, Hyowon Gweon, C. Karen Liu, Jiajun Wu, and Li Fei-Fei. 2024. BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation. arXiv:2403.09227 [cs.RO] <https://arxiv.org/abs/2403.09227>
- [6] Dacheng Li, Yunhao Fang, Yukang Chen, Shuo Yang, Shiyi Cao, Justin Wong, Michael Luo, Xiaolong Wang, Hongxu Yin, Joseph E. Gonzalez, Ion Stoica, Song Han, and Yao Lu. 2025. WorldModelBench: Judging Video Generation Models As World Models. <https://huggingface.co/papers/2502.20694>. Accessed: 2026-05-18.
- [7] Yaxuan Li, Zhongyi Zhou, Yefei Chen, Yaokai Xue, and Yichen Zhu. 2026. dWorldEval: Scalable Robotic Policy Evaluation via Discrete Diffusion World Model. arXiv:2604.22152 <https://arxiv.org/abs/2604.22152>
- [8] Yaxuan Li, Yichen Zhu, Junjie Wen, Chaomin Shen, and Yi Xu. 2025. WorldEval: World Model as Real-World Robot Policies Evaluator. arXiv:2505.19017 [cs.RO] <https://arxiv.org/abs/2505.19017>
- [9] Soroush Nasiriany, Abitha Thankaraj, Zhecheng Yuan, Abhijeet Gupta, Li Fei-Fei, and C. Karen Liu. 2024. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. arXiv:2406.02523 [cs.RO] <https://arxiv.org/abs/2406.02523>
- [10] NVIDIA. 2025. NVIDIA Announces Isaac GR00T N1, an Open Humanoid Robot Foundation Model and Simulation Frameworks. <https://nvidia.com/news/nvidia-isaac-gr00t-n1-open-humanoid-robot-foundation-model-simulation-frameworks>. Accessed: 2026-05-18.
- [11] Physion Labs. 2025. Physion Atlas 1.0. <https://physionlabs.ai/blog/physion-atlas1.0>. Accessed: 2026-05-18.
- [12] RoboCasa365 Authors. 2026. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. arXiv:2603.04356 [cs.RO] <https://arxiv.org/abs/2603.04356>
- [13] Lun Wang. 2025. Your Evals Will Break. <https://wanglun1996.github.io/blog/your-evals-will-break.html>. Accessed: 2026-05-18.
- [14] WorldArena Team. 2026. WorldArena: A Unified Benchmark for Evaluating Perception and Functional Utility of Embodied World Models. <https://world-arena.ai/>. Accessed: 2026-05-18.
- [15] Chenyu Zhang, Daniil Cherniavskii, Andrii Zadaianchuk, Antonios Tragoudaras, Antonios Vozikis, Thijmen Nijdam, Derck W. E. Prinzhorn, Mark Bodracska, Nicu Sebe, and Efstratios Gavves. 2025. Morpheus: Benchmarking Physical Reasoning of Video Generative Models with Real Physical Experiments. <https://huggingface.co/papers/2504.02918>. Accessed: 2026-05-18.
- [16] Qin Zhang, Peiyu Jing, Hong-Xing Yu, Fangqiang Ding, Fan Nie, Weimin Wang, Yilun Du, James Zou, Jiajun Wu, and Bing Shuai. 2026. Physion-Eval: Human Reasoning Traces for Evaluating Physical Realism in Generated Videos. arXiv:2603.19607 <https://arxiv.org/abs/2603.19607>